An Introduction to E-Commerce

with a Focus on Machine Learning, Deep Learning, and Artificial Intelligence

Lecture 05: Enterprise Integration Patterns for E-Commerce

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences
Mansoura University
Egypt

March 23, 2026

Session plan (90 minutes)

1. Why integration patterns matter
2. Integration styles
3. Choosing the right integration style
4. API-first integration
5. Idempotency
6. Hands-on lab: modeling an order lifecycle

Learning objectives

By the end of this lecture, you should be able to:

- Compare synchronous APIs, asynchronous events, and batch integrations in enterprise commerce environments.
- Explain why reliability patterns such as idempotency, retries, dead-letter handling, and the outbox pattern are essential in distributed commerce systems.
- Design integration contracts that reduce coupling between storefront, ERP, payment, logistics, and data platforms.
- Evaluate integration choices in terms of latency, resilience, consistency, and operational complexity.

Why integration patterns matter

- E-commerce systems rarely operate as a single application.
- Even a modest commerce stack may involve a storefront, search service, promotions engine, payment provider, ERP, warehouse system, shipping partner, CRM, and analytics platform.
- These systems must exchange data continuously while still tolerating delays, retries, outages, and partial failures.

Integration styles

- **Synchronous request-response:** payment authorization during checkout,; tax estimation,
- **Asynchronous events:** `OrderCreated`,; `InvoicePosted`,
- **Batch and file-based integration:** latency requirements are low,; partner systems are legacy or externally controlled,

Choosing the right integration style

- **Questions to ask:** Does the user or operator need an immediate answer?; What is the tolerance for stale data or delayed propagation?
- **Typical choices in commerce: Checkout payment authorization:** usually synchronous because the customer needs immediate confirmation.; **Order export to ERP:** often asynchronous because the storefront should not block on internal back-office processing.

API-first integration

- **Good API design principles:** Model APIs around business resources and actions, not internal database tables.; Make required fields, validation rules, and error semantics explicit.
- **Commands versus queries: Queries:** read current information, such as "get available shipping options."; **Commands:** request a state change, such as "create order" or "capture payment."

Idempotency

- **Why idempotency is required:** Networks fail in ambiguous ways.
- **Idempotency keys:** making the key stable for a business attempt,; scoping it to the operation type,

Events and event-driven architecture

- **Domain events versus technical events: Domain events:** meaningful business facts such as `OrderCanceled`.; **Technical events:** operational notifications such as cache refresh or job completion.
- **Benefits of event-driven integration:** Producers and consumers can evolve more independently.; Multiple downstream systems can react to the same fact.
- **Costs of event-driven integration:** Debugging becomes harder because workflows are distributed across time and services.; Consumers must tolerate reordering, duplication, and replay.

Reliability patterns in distributed workflows

- **Retries and backoff:** Transient failures are normal.
- **Dead-letter handling:** If a message repeatedly fails processing, it should not block the entire flow indefinitely.
- **Timeouts and circuit breakers:** temporarily disabling a non-critical recommendation widget,; falling back to cached delivery promises,
- **Reconciliation jobs:** Not all inconsistencies can be prevented in real time.

The outbox pattern

- **How the outbox pattern works:** writing the business state change and an "outbox record" in the same local transaction,; then having a separate relay process publish the outbox record to the event broker,
- **Why it matters in commerce:** Commerce workflows are especially sensitive to dual-write failures because duplicated or missing side effects can affect revenue, customer trust, and financial records.

Message contracts and schema governance

- **What a contract should define:** message or API name,; required and optional fields,
- **Backward compatibility:** adding optional fields rather than repurposing existing ones,; preserving existing semantics across versions,

Observability for integrations

- **Minimum signals: Logs:** structured records with correlation IDs, idempotency keys, and error codes.; **Metrics:** request rates, latency, failure rates, retry counts, dead-letter counts, consumer lag.
- **Correlation and traceability:** Cross-system workflows should carry shared identifiers such as order ID, correlation ID, and experiment or campaign context where relevant.

iPaaS, ESB, brokers, and workflow orchestration

- **iPaaS and ESB concepts: ESB-style platforms:** historically emphasized centralized mediation, transformation, and routing.; **iPaaS platforms:** emphasize managed connectors, workflow automation, and cloud integration convenience.
- **Event brokers and streams:** Message brokers and streaming platforms provide transport for asynchronous communication.
- **Workflow orchestration:** Some workflows are easier to manage with explicit orchestration, especially long-running processes such as returns handling, supplier exceptions, or manual fraud review.

Integration anti-patterns

- direct database-to-database coupling across domains,
- duplicating business rules independently in multiple systems,
- publishing events without stable schemas or owners,
- assuming exactly-once delivery where only at-least-once semantics are realistic,
- using synchronous calls for every workflow step,
- and treating reconciliation as proof that real-time design can be ignored.

Hands-on lab: modeling an order lifecycle

- **Lab scenario:** Assume a headless storefront, a payment service, an order service, Odoo, and a shipment service.
- **Lab tasks:** Identify which interactions are synchronous and which are asynchronous.; Define an idempotency strategy for order creation and payment capture.
- **Expected learning outcome:** By the end of the lab, students should be able to explain integration design in terms of business guarantees rather than technology labels alone.

Managerial and architectural implications

- synchronous calls where immediate business decisions are required,
- events where decoupling and fan-out matter,
- batch where latency is not critical,
- and governance across all three.